TECHNICAL REPORT

Survival Model Evaluation: chicago_licences

Fitted with the strategy-survival-model pipeline; methodology validated separately against synthetic ground truth

Source data	`datasets/chicago_licences.csv.gz`
Rows	262,763 (83.6% with the ending observed, 16.4% censored)
Start dates	2002-01-02 to 2026-07-31
Evaluation	5 expanding-window temporal folds
Source of figures	`reports/chicago_demo/metrics.json`, regenerated by the command in section 7

---

1. Summary

This report evaluates a survival model fitted to `chicago_licences.csv.gz`, 262,763 rows, each observed from its start date for 'licensed_days' days with 'closed' marking whether the ending was seen (83.6%) or the row was still running when observation stopped (16.4%). Median observed duration is 759 days. The model predicts, from the 300 features available at the start date, how long each row survives.

Pooled over 157,658 out-of-time test rows, the XGBoost AFT model reaches a concordance index of 0.697 (95% bootstrap interval 0.695 to 0.699), against 0.500 for a coin flip. Set beside a Cox proportional hazards baseline on the same features it scores 0.695 to the baseline's 0.695; both of those are fold means, which is the only like-for-like basis and is why the pooled figure is not the one compared. At the 1095-day horizon its censoring-weighted Brier score is 0.206, beating the 0.247 of a no-skill forecast that assigns every row the population average.

That bootstrap interval is narrow, and it is worth saying what it does and does not cover. It holds each fold's fitted model fixed and resamples the test rows, so it measures how precisely this run's score is pinned down by the number of rows scored, nothing more. It says nothing about how much the score would move on a different stretch of history, and the per-fold spread of 0.604 to 0.782 in Table 2 is the better guide to that. The rows are also unlikely to be independent, since rows that share a category or a stretch of time tend to fail together, which makes the true interval wider than the one printed.

> This dataset has no known generating process. Unlike the synthetic validation report, there is no oracle ceiling to say how much signal remains unclaimed, and feature attributions cannot be checked against a true mechanism. What carries over from the synthetic validation is the pipeline itself: the same temporal folds, label re-censoring, likelihood-based selection, and calibration checks, verified there against known answers.

2. Data

The dataset is `datasets/chicago_licences.csv.gz`: 262,763 rows, each observed from its start date. Start dates run 2002-01-02 to 2026-07-31. 83.6% of rows have their ending observed and 16.4% are censored; median observed duration is 759 days. Text columns one-hot encoded via the saved encoding recipe: ward, community_area, police_district, zip_code.

Figure 1 shows Kaplan-Meier survival curves by `license_description`, the coarsest structure in the outcome before any model is fitted.

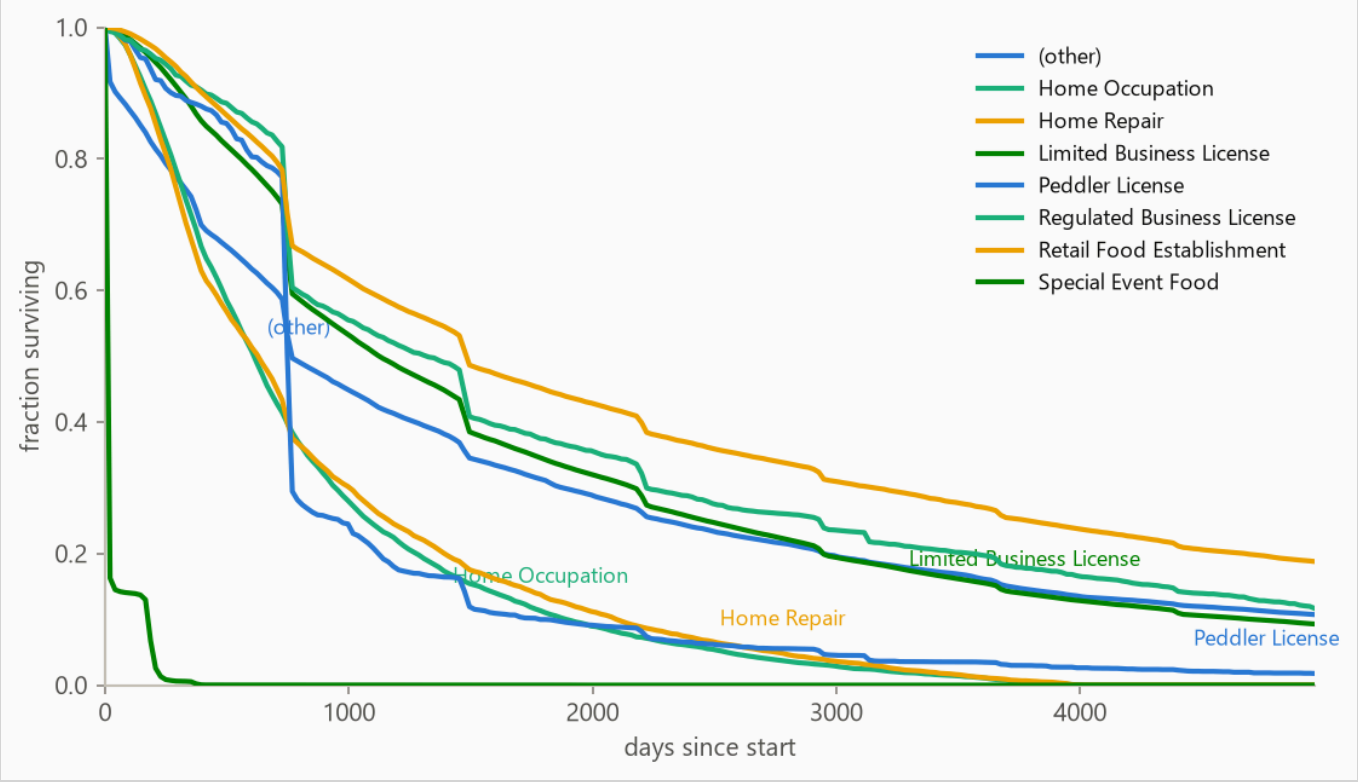

Figure 1. Kaplan-Meier survival curves by `license_description`. Groups beyond the seven most frequent are collapsed into (other) for this plot only.

3. Method

3.1 Model class

The target is a duration with incomplete observations, so the model is an accelerated failure time (AFT) model, implemented as XGBoost with the `survival:aft` objective. Censoring enters through interval labels, where an observed death is the interval [t, t] and a censored row is [t, infinity).

Regression on lifetime was rejected because censored rows have no target, and dropping them discards the longest-lived rows, which biases the model toward pessimism. Classification at a fixed horizon handles censoring awkwardly and answers only one question. AFT keeps every row and returns a full time distribution, which collapses to any horizon an allocator asks about.

Concretely, the fitted model predicts one number per row, the median survival time in days. The full curve comes from wrapping a log-normal distribution of fitted width around that median, so the probability of surviving any horizon is a point read off that curve. The width is a single value shared by every row; how it is selected and calibrated is covered in section 3.3.

Numeric features pass through as-is, missing values included, which XGBoost handles natively; the Cox baseline receives train-window median imputation. Text columns are one-hot encoded.

3.2 Temporal validation and label re-censoring

Rows are ordered by start date and evaluated with 5 expanding-window folds. The earliest 40% of rows is burn-in that is only ever trained on. Every fold trains only on rows that started before its split date and tests on the next block, so training sets grow from 105,103 to 231,220 rows.

Training labels are re-censored at each split date. A row started two years before a split may have died three months after it, and its recorded label contains that future death. Any model trained at that split date could only have known the row was still running, so every post-split death is rewritten as a censoring at the split. Omitting this step raises measured scores while silently importing future information, which makes it the most consequential detail in the pipeline. It is implemented as a standalone function (`cv.recensor`) with dedicated tests, because it transfers unchanged to any duration problem on operational data.

Test labels use the full observation window, because the restriction exists to keep the future away from the model, not away from the scorer.

3.3 Hyperparameter selection and calibration

Hyperparameters are selected once, on the first fold's training window, using an inner temporal split and held-out censored log-likelihood. Likelihood rather than concordance drives selection, and the reason is a failure this pipeline recorded rather than avoided.

An earlier version selected on concordance and looked correct: ranking metrics landed where they ultimately did. It then lost to a no-skill reference forecast on 365-day Brier score. A concordance index is invariant to the predictive scale, so the search had no reason to prefer a usable distribution width and settled on one far too wide. Nothing in the training loop objected, because nothing in the training loop was measuring it. Selection now uses likelihood, which penalizes a broken scale, and the predictive log-normal scale is fitted separately on a temporal tail slice the early-stopping probe never trained on. The selected loss scale was 0.9, and the calibrated predictive scale came out at 1.28.

The general lesson holds beyond this project. A model can rank well and misstate probabilities at the same time, and the only defense is evaluating both.

3.4 Baselines

A Cox proportional hazards model fitted on the same features is the standard survival alternative and answers whether the gradient-boosted model was necessary.

4. Results

4.1 Discrimination

Pooled over 157,658 out-of-fold rows, with 95% percentile bootstrap intervals over 500 resamples:

Table 1. Concordance index by method, pooled across folds. Higher is better; 0.500 is a coin flip.

METHOD	HARRELL C	95% INTERVAL
XGBoost AFT (pooled)	**0.697**	**0.695 to 0.699**
XGBoost AFT (fold mean)	0.695	not resampled
Cox proportional hazards (fold mean)	0.695	not resampled

On this dataset the Cox baseline outscores the boosted model, 0.695 against 0.695 on fold-mean concordance. Both fitted models are saved and the Cox model is the one this run recommends for scoring new rows. Reporting that, rather than presenting the boosted model as the result, is the point of carrying a baseline at

all. Each fold refits the Cox model, and its risk scores are relative ones centred on that fold's own training window, so they are meaningful within a fold but carry no common scale across folds; the fold-mean rows are the like-for-like comparison and the pooled figure is not comparable to them.

Table 2. Per-fold results. Censoring is the share of each fold's test rows whose ending was not observed.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX
1	2009-11-09	105,103	31,532	6%	0.701	0.695
2	2012-08-02	136,592	31,532	14%	0.604	0.617
3	2014-06-09	168,159	31,532	16%	0.707	0.706
4	2017-10-23	199,693	31,531	27%	0.680	0.672
5	2022-02-23	231,220	31,531	60%	0.782	0.787

Per-fold stability is the claim Figure 2 supports; Table 2 carries the same numbers with their censoring mix.

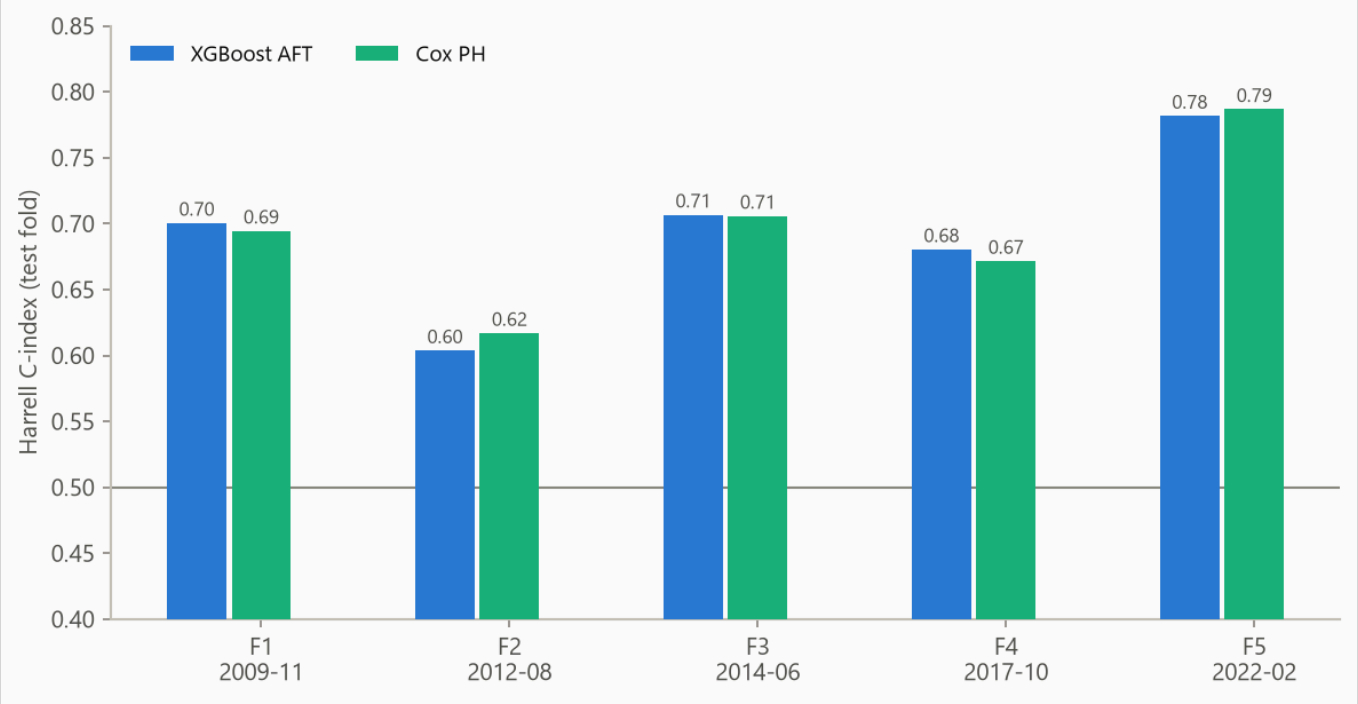

Figure 2. Concordance by fold, from 0.604 to 0.782. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

4.2 Calibration

Predicted median survival times are converted to survival probabilities under the calibrated log-normal, then checked two ways. Brier scores are weighted by the inverse probability of censoring so that censored rows do

not bias the result. The no-skill reference assigns every row the pooled Kaplan-Meier marginal, ignoring all features.

Table 3. IPCW Brier score by horizon. Lower is better.

HORIZON	AFT	COX	NO-SKILL MARGINAL
365 days	0.084	0.082	0.158
1095 days	0.206	0.207	0.247
1825 days	0.194	0.192	0.215

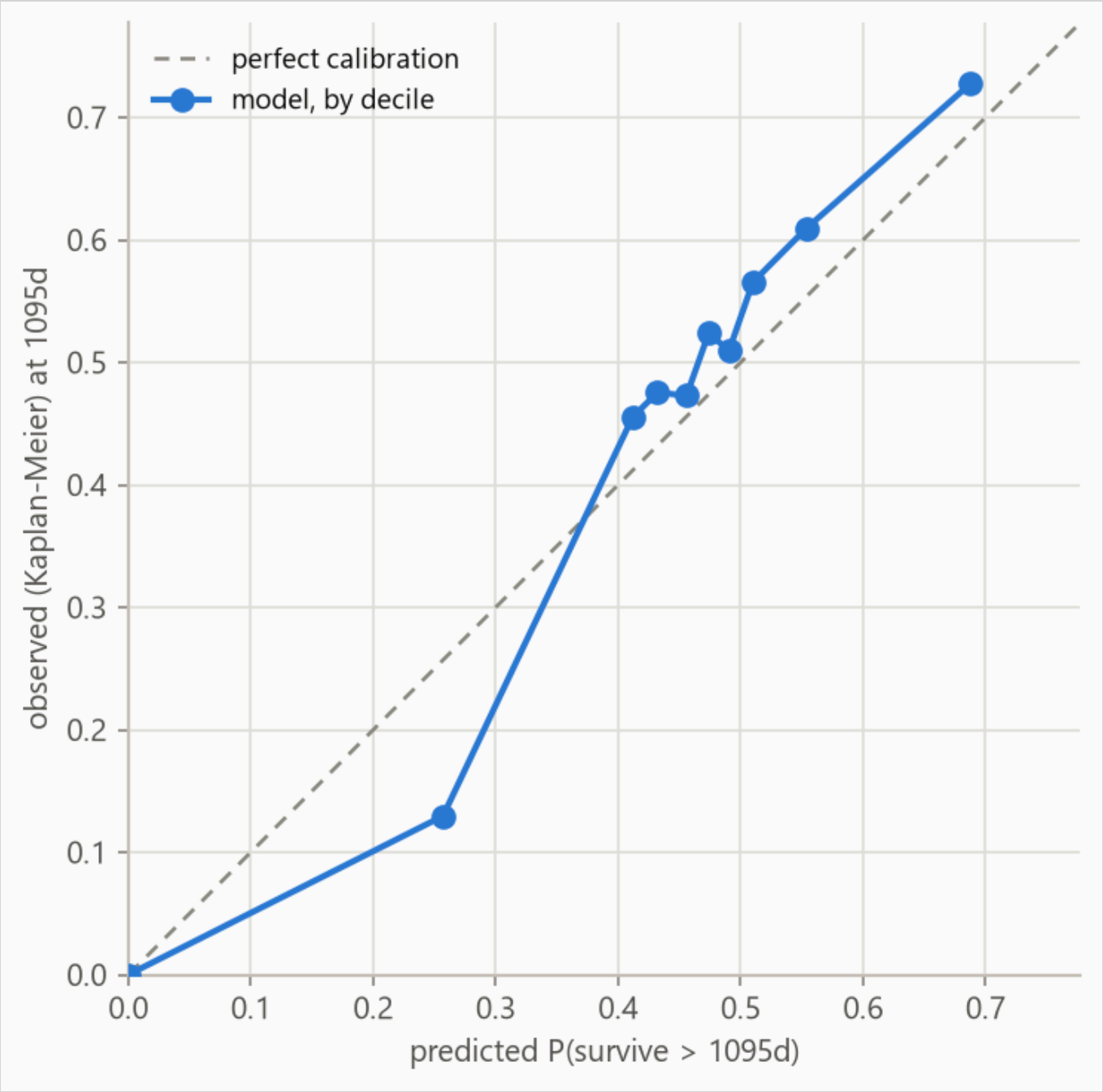

Figure 3. Predicted against observed survival at 1095 days, by predicted decile. Observed frequencies are Kaplan-Meier estimates within each bin, so censored rows contribute correctly.

Table 4. Decile calibration at 1095 days, the values plotted in Figure 3.

DECILE	N	PREDICTED	OBSERVED (KM)	DEVIATION
1	15797	0.000	0.000	-0.000
2	15748	0.257	0.130	-0.127
3	16166	0.412	0.455	+0.044
4	15480	0.432	0.475	+0.043
5	17372	0.456	0.473	+0.017
6	14036	0.474	0.524	+0.050
7	15761	0.490	0.509	+0.019
8	16054	0.510	0.566	+0.055
9	15601	0.553	0.609	+0.056
10	15643	0.688	0.728	+0.040

5. What the model uses

Feature attributions are computed on the log-time margin, so a value of +0.3 multiplies predicted survival time by about 1.35 and negative values shorten it.

On this data the attributions are descriptive only. There is no known mechanism to check them against, and correlated features can split credit in ways that reflect the model's internal choices as much as the data, so read the ranking as "what this model leaned on", not as importance in the world.

Figure 4 ranks features by mean absolute attribution, Table 5 lists the top eight, Figure 5 shows the per-row spread, and Figure 6 traces attribution against feature value.

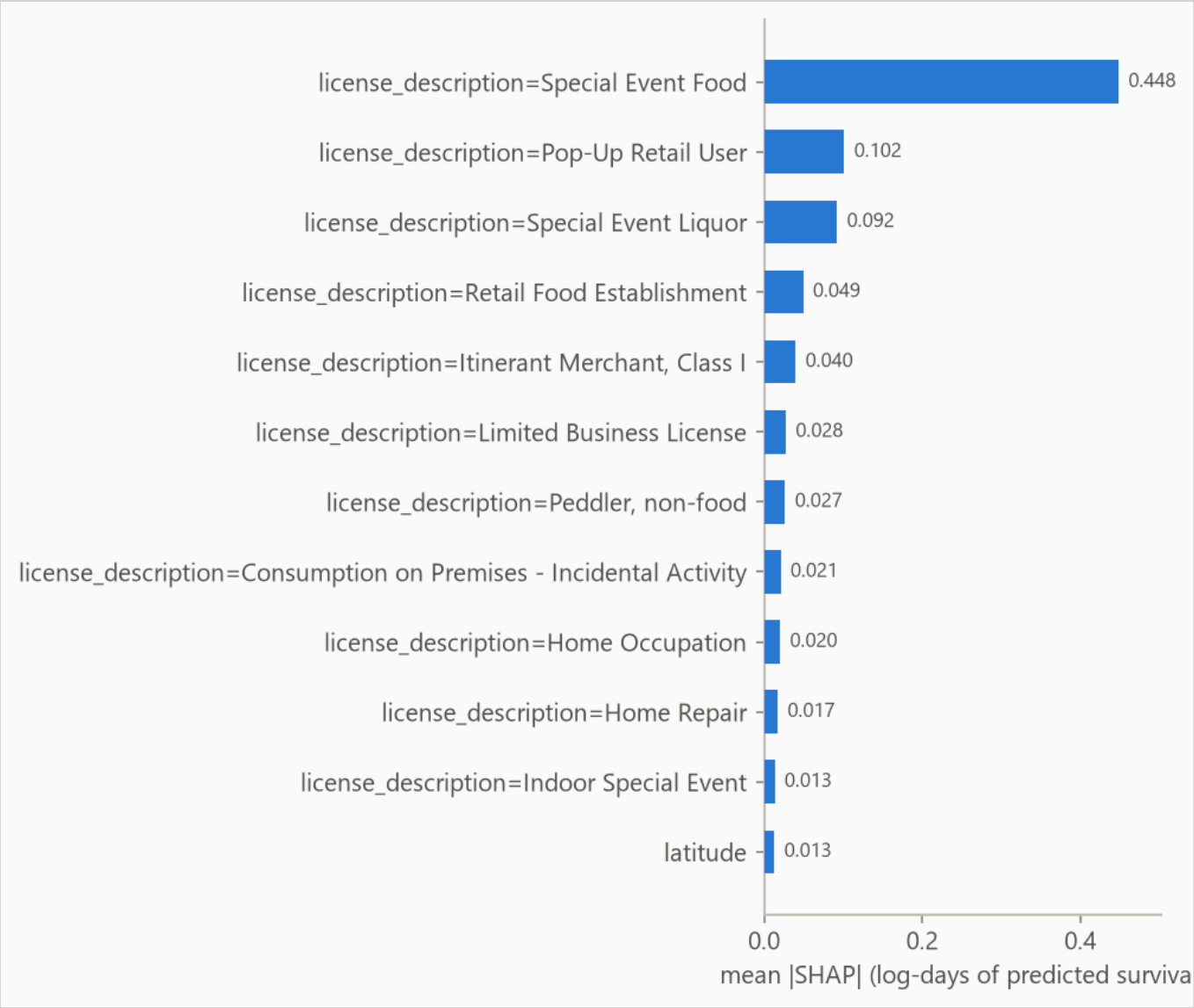

Figure 4. Mean absolute attribution by feature.

Table 5. Top eight features by mean absolute attribution.

FEATURE	MEAN |ATTRIBUTION|
license_description=Special Event Food	0.448
license_description=Pop-Up Retail User	0.102
license_description=Special Event Liquor	0.092
license_description=Retail Food Establishment	0.049
license_description=Itinerant Merchant, Class I	0.040
license_description=Limited Business License	0.028
license_description=Peddler, non-food	0.027
license_description=Consumption on Premises - Incidental Activity	0.021

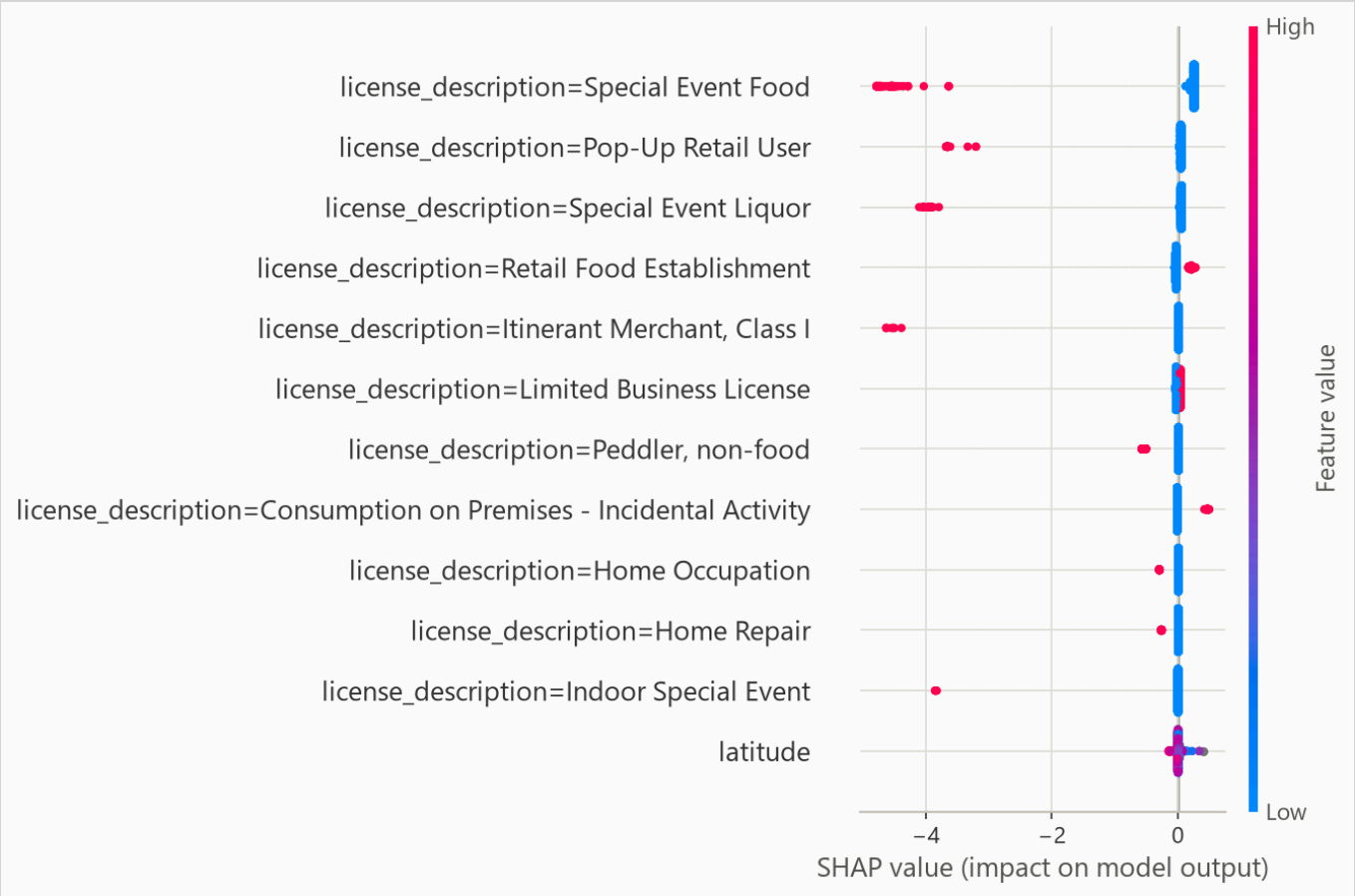

Figure 5. Per-row attributions across the explanation sample.

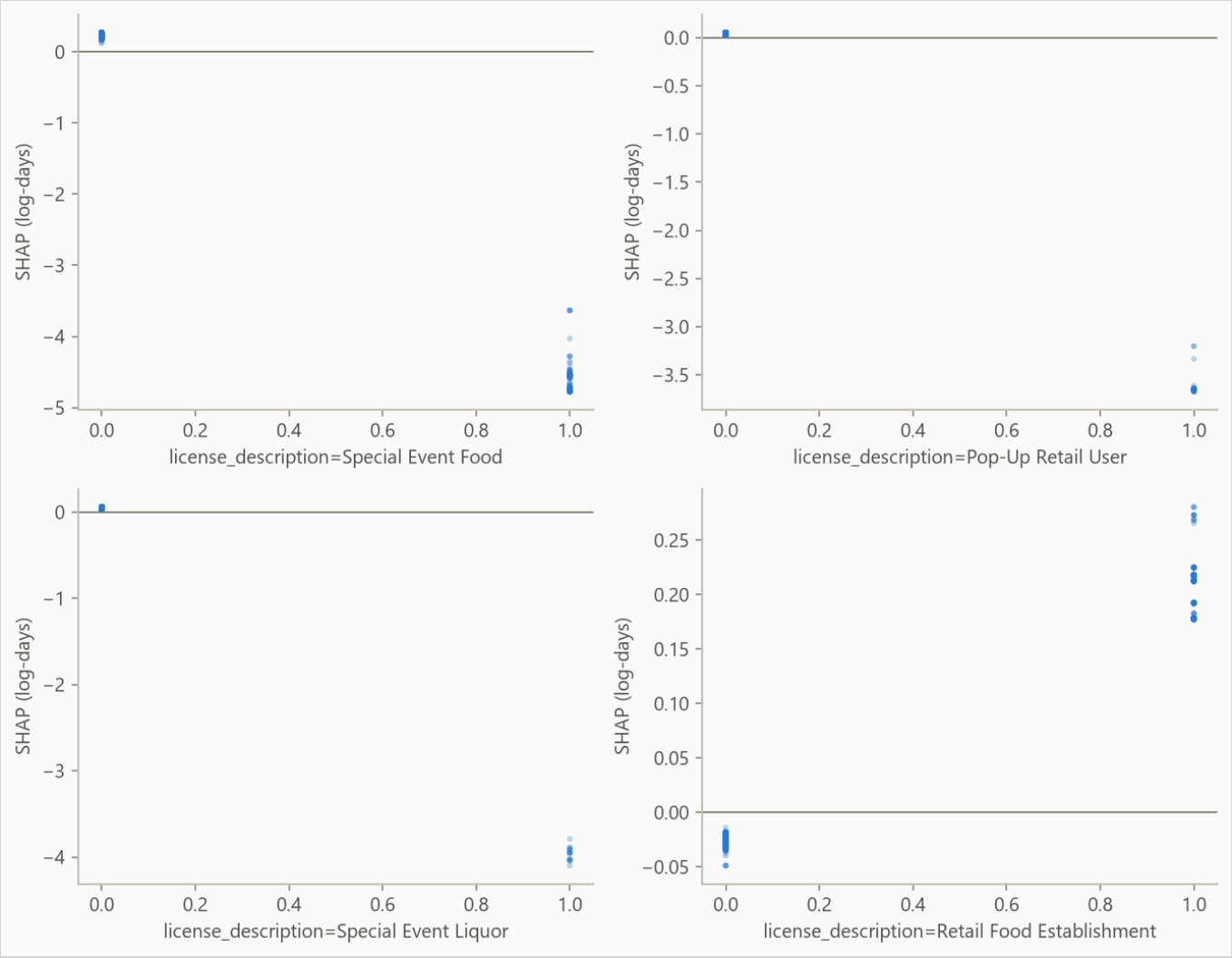

Figure 6. Attribution against feature value for the four features with the largest mean attribution.

6. Limitations

No ground truth. Nothing bounds how much predictable signal this dataset contains, so the concordance above cannot be placed relative to a ceiling the way the synthetic validation could.

Censoring may be informative. The evaluation assumes rows stop being observed for reasons unrelated to their risk. If observation ended early on rows that were about to end anyway, survival estimates are biased upward. Whether that holds here depends on how this dataset was collected.

Every row shares one curve shape. The predictive scale is a single fitted number, so the model shifts each row's survival curve earlier or later but never widens or narrows it, and two rows with the same predicted median receive identical probabilities at every horizon. Letting the width vary by row would need a model that predicts it, which is future work.

Harrell's concordance is biased under heavy censoring. The final fold is 60% censored, where an inverse-probability weighted concordance would be preferable. Fold-to-fold comparisons in Table 2 should be read with that in mind.

7. Reproducing this run

```
python scripts/run_fit_evaluate.py --data datasets/chicago_licences.csv.gz --name chicago_licences
python scripts/run_build_report.py --run reports/chicago_demo
```

Generated from `reports/chicago_demo/metrics.json` by `scripts/run_build_report.py --run`. 262,763 rows, 157,658 out-of-time test rows.